

Principles of Programming Languages:

Problem Sheet 2 - Memory and monads

Question 1

```
1  -- Q1
2  val x1 = new();;
3  val y1 = x1;;
4
5  val x2 = new();;
6  val y2 = new();;
7
8  val f(x,y) =
9      x := 1; y := 2;
10     !x = !y;;
11
12 f(x1, y1);;
13 --> true
14 f(x2, y2);;
15 --> false
```

Question 2

```
1  -- Q2
2  val a = new();;
3  val b = new();;
4
5  b := a;;
6  !b = a;;
7  --> true
8  a := 2;;
9  !!b;;
10 --> 2
11
12 val c = new();;
13 val clst = 1:c;;
14 c := clst ;;
15 clst ;;
16 --> [1 . <address 3>]
17 !tail( clst );;
18 --> [1 . <address 3>]
```

Question 3

```
1 -- Q3
2 eval :: Expr -> Env -> Mem -> (Value, Mem)
3 eval (Repeat e1 e2) env mem = f mem
4   where
5     f mem =
6       let (a, mem') = eval e1 env mem in
7       let (b, mem'') = eval e2 env mem' in
8       case b of
9         BoolVal False -> f mem''
10        BoolVal True  -> (Nil, mem'')
11        _              -> error "boolean required in expression 2"
```

Question 4

```
1 -- Q4
2 primitive "put" (\ [Addr a, b] mem -> (Nil, update mem a b))
```

Question 5

```
1 -- Q5
2 -- Suppose xm = \mem -> (x, mem), f x = \mem -> (y, mem), g y = \mem -> (z, mem)
3 (xm $> f) $> g
4 = \mem -> let (x, mem') = xm mem in f x mem' $> g
5 = \mem -> let (x, mem') = xm mem in let (y, mem'') = f x mem' in g y mem''
6 xm $> (\x -> f x $> g)
7 = xm $> \x -> (\mem -> let (x', mem') = x mem in let (y, mem'') = f x' mem' in g y mem'')
8 = \mem -> let (x', mem') = xm mem in let (y, mem'') = f x' mem' in g y mem''
9
10 result x $> f
11 = \mem -> (x, mem) $> f
12 = \mem -> let (y, mem') = x mem in f y mem'
13 = \mem -> f x mem
14 = f x
15
16 xm $> result
17 = \mem -> let (x', mem') = xm mem in result x' mem'
18 = \mem -> let (x', mem') = (x, mem) in result x' mem'
19 = \mem -> result x mem
20 = \mem -> (x, mem)
21 = xm
```

Question 6

```
1 module MonadMemPrint where
2 import Data.List
3
4 type Value = Int
5
6 infixl 1 $>
7
8 type M a = [Value] -> (String, a, [Value])
9 type Location = Int
10
11 result :: a -> M a
12 result x = \mem -> ("",x,mem)
13
14 ($>) :: M a -> (a -> M b) -> M b
15 ($>) xm f =
16   \mem -> let (s,x,mem') = xm mem in let (t,y,mem'') = f x mem' in (s ++ t, y, mem'')
17
18 put :: Location -> Value -> M ()
19 put a v = \mem -> ("", (), (take a mem) ++ v : (drop (a+1) mem))
20
21 get :: Location -> M Value
22 get a = \mem -> ("", mem !! a , mem)
23
24 new :: M(Location)
25 new = \mem -> ("",length mem,mem++[0])
26
27 output :: String -> M()
28 output s = \mem -> (s, (), mem)
29
30 test = new $> \a ->
31   put a 40 $> \_ ->
32   get a $> \v ->
33   put a (v+1) $> \_ ->
34   output "hello " $> \_ ->
35   output "world" $> \_ ->
36   get a $> \v ->
37   result (v+1)
38 -- test []
39 -- ("hello world",42,[41])
```

Question 7

```

1  val x = new();;
2  x := 0;;
3
4  val a = (lambda () x:= 1; !x): (lambda () x:= 2; !x): (lambda () !x): nil ;;
5
6  val g = head(a);;
7  val h = head(tail(a));;
8  val i = head(tail(tail(a)));;
9
10 val f(w, y, z) =
11     x := 3;
12     y:!x: nil ;;
13
14 -- x is set by g(), then i() outputs x, then x is set by h()
15 f(g(), i(), h());;
16 --> [1, 2]
17 x := 0;;
18 -- x is set by h(), then i() outputs x, then x is set by g()
19 f(h(), i(), g());;
20 --> [2, 1]
21 -- Thus, arguments are evaluated left to right
22
23 -- function objects are evaluated first , not sure how to show

```

(b) Update `evalargs` to evaluate parameters right to left, and `eval (Apply f es)` to evaluate the function after.

```

1  -- Q7(b)
2  eval :: Expr -> Env -> (Value -> Answer) -> Answer
3
4  eval (Apply f es) env k =
5      evalargs es env (\ args ->
6          eval f env (\ fv ->
7              apply fv args k))
8
9  evalargs :: [Expr] -> Env -> ([Value] -> Answer) -> Answer
10 evalargs [] env k = k []
11 evalargs (e:es) env k =
12     evalargs es env (\ vs -> eval e env
13         (\v -> k (v:vs)))

```

Question 8

```
1  (e_1; e_2); e_3
2  = e_1; ((lambda x -> e_2 x); e_3)
3  = e_1; ((lambda x -> e_2 x); e_3; result )
4  = e_1; ((lambda x -> e_2 x); (lambda y -> e_3 y; result))
5  = e_1; ((lambda x -> e_2 x); (lambda y -> e_3 y))
6  = e_1; (lambda x -> e_3 (e_2 x))
7  = e_1; (e_2; e_3)
```

```
1  nil ; e1
2  = (nil ; result ); e1
3  = nil ; (lambda x -> result x; e1)
4  = nil ; (lambda x -> e1 x)
5  = e1
```

`nil` may not satisfy the type of the monad.

```
1  e1; nil
2  = (e1; result ); nil
3  = e1; (lambda x -> result x; nil )
4  = e1; (lambda x -> nil x)
5  = nil
```

(b) From Tutorial 1, we know

```
1  let val x = e1 in e2
2  = (lambda (x) e2)(e1)
```

Then it is shown that the lambda expression is equivalent.

```
1  e1; e2
2  = (e1; e2); result
3  = e1; (lambda x -> e2 x; result)
4  = e1; lambda x -> e2 x;
5  = (lambda x -> e2)(e1)
```

Question 9

```
1  -- Q9
2  mapm :: (a -> M b) -> [a] -> M [b]
3  mapm f [] = result []
4  mapm f (e:es) =
5    f e $> \v ->
6    mapm f es $> \vs ->
7    result (v:vs)
```

Question 10

```
1  -- Q10
2  mmap :: (a -> b) -> (M a -> M b)
3  mmap f = \xm -> xm $> \a -> result (f a)
4
5  join :: M (M a) -> M a
6  join mma = mma $> \ma -> ma
7
8  mmap id
9  = \xm -> xm $> \a -> result (id a)
10 = \xm -> xm $> \a -> result a
11 = \xm -> xm $> result
12 = \xm -> xm
13 = id
14
15 mmap g . mmap f
16 = \xm -> mmap f xm $> \a -> result (g a)
17 = \xm -> xm $> \a' -> result (f a') $> \a -> result (g a)
18 = \xm -> xm $> \a' -> result (g (f a'))
19 = mmap (g . f)
20
21 mmap f . result
22 = \x -> result x $> \a -> result (f a)
23 = \x -> result (f x)
24 = result . f
25
26 join . mmap (mmap f)
27 = \xmm -> mmap (mmap f) xmm $> \xm -> xm
28 = \xmm -> xmm $> \xm' -> result ((mmap f) xm') $> \xm -> xm
29 = \xmm -> xmm $> \xm' -> result (xm' $> \x -> result (f x)) $> \xm -> xm
30 = \xmm -> xmm $> \xm' -> (xm' $> \x -> result (f x))
31 = (\xmm -> xmm $> \xm' -> xm') $> \x -> result (f x)
32 = (\xmm -> join xmm) $> \x -> result (f x)
33 = mmap f . join
34
35 join . mmap join
36 = \xmm -> mmap join xmm $> \xm -> xm
37 = \xmm -> xmm $> \xm -> result (join xm) $> \xm -> xm
38 = \xmm -> xmm $> \xm -> join xm
39 = \xmm -> xmm $> \xm -> xm $> \x -> x
40 = \xmm -> join xmm $> \x -> x
41 = join . join
42
43 join . result
44 = \xmm -> result xmm $> \xmm' -> xmm'
45 = \xmm -> xmm
46 = id
47
48 join . mmap result
49 = \xmm -> mmap result xmm $> \xmm' -> xmm'
50 = \xmm -> xmm $> \a -> result (result a) $> \xmm' -> xmm'
51 = \xmm -> xmm $> \a -> result a
52 = \xmm -> xmm
```

$$_{53} = \mathbf{id}$$